

**Implementing Snort IDS to secure  
DVWA  
Against Common Cyber Attacks  
Part - 2**

By : J Saiprakash  
- JUNE 2026

As part of this self-initiated project, this phase focused on exploring Snort's preprocessor-based detection capabilities. Multiple attack simulations were performed to study how Snort detects protocol anomalies, network reconnaissance, denial-of-service activity, and evasion techniques through traffic normalization and behavioral analysis.

The environment was set up on a single Ubuntu VM, configured with a dual-interface network setup to better handle traffic and reduce false positives. This setup allowed me to gain practical, hands-on experience in writing and tuning Snort rules, and helped me understand how custom detection can be applied to simulate real-world attack scenarios in a controlled lab environment.

## What is DVWA?

DVWA stands for Damn Vulnerable Web Application. It is a web application designed with known security vulnerabilities. It allows security professionals and enthusiasts to practice identifying and exploiting common web application weaknesses in a controlled and safe environment. DVWA provides a platform to study security concepts and test the effectiveness of security tools.

Deployed DVWA using the official GitHub repository:

<https://github.com/digininja/DVWA>

## What is Snort?

SNORT is a powerful open-source Intrusion Detection System (IDS) and Intrusion Prevention System (IPS) that provides real-time network traffic analysis and data packet logging. SNORT uses a rule-based language that combines anomaly, protocol, and signature inspection methods to detect potentially malicious activity.

### IDS:

IDS is an passive monitoring solution for detecting possible malicious activities/patterns ,abnormal incidents, and policy violations.

It is responsible for generating alerts for each suspicious event.

### IPS:

IPS is an active inline monitoring solution for detecting possible malicious activities/patterns, abnormal incidents, and policy violations.

It is responsible for stopping/preventing/terminating the suspicious event as soon as the detection is performed.

# What are Snort Rules?

Snort rules are user-defined instructions that tell the Snort Intrusion Detection System (IDS) and Intrusion Prevention System (IPS) what kind of network traffic to watch for and how to respond when it is detected. These rules are written in a specific syntax that allows Snort to inspect packets in real-time and trigger alerts(IDS mode) or block malicious traffic(IPS mode).

Snort alerts come from **TWO SOURCES**:

- 1) Preprocessor-generated alerts
- 2) Rule-based alerts

(Snort alerts are generated either by detection rules or by internal preprocessor modules that detect protocol-level anomalies.)

## Preprocessor-generated alerts:

**Preprocessor-generated alerts** are alerts raised **internally by Snort preprocessors** when they detect **protocol violations, anomalies, or evasion techniques, without using pattern-matching rules**. Snort preprocessors apply normalization to network traffic before rule evaluation.

## Normalization in Snort

Normalization in Snort is the process of cleaning, decoding, and rebuilding network traffic so that hidden or obfuscated attacks become visible to detection rules.

Before Snort checks traffic against detection rules, it first uses preprocessors to assemble, decode, and normalize the packets. The cleaned traffic is then sent to the detection engine where alert classification is applied.

Snort includes a default normalization configuration that is enabled through its built-in preprocessors and defined in the **snort.conf** file. These settings do not need to be created manually, but they can be modified or tuned if required.

Snort preprocessors operate across multiple layers of the network stack, starting from the Internet (IP) layer up to the application layer.

Each layer includes different preprocessors that generate specific events and apply normalization before detection rules are evaluated. **In this project, only selected preprocessors are covered.**

At the **IP layer**, normalization is handled by the **frag3 preprocessor**, which reassembles fragmented IP packets and detects fragmentation-based evasion.

At the **Transport layer**, normalization is handled by the **stream5 preprocessor**, which rebuilds TCP sessions and tracks UDP traffic.

At the **Application layer**, normalization is performed by application-specific preprocessors. In this lab, HTTP traffic is used, so the **http\_inspect preprocessor** is covered for HTTP normalization and anomaly detection.

The core idea of **normalization** is:

**Make different representations of the same attack look identical before detection.**

Without normalization, an IDS sees packets exactly as they arrive. Attackers can exploit this by changing the way the attack is written while keeping the same meaning.

## frag3 preprocessor.

The frag3 preprocessor works at the IP layer and is responsible for handling IP fragmentation and detecting overlapping IP fragment attacks.

Attackers often use IP fragmentation as an evasion technique by splitting a malicious payload across multiple IP fragments so that no single fragment contains the full attack. Attackers may also use overlapping IP fragments to confuse IDS detection.

When these fragments reach the destination system, they are reassembled into the original IP packet using information in the IP header.

If an IDS inspects each fragment separately without reassembling them, the complete attack payload may not be detected.

## Default Snort Preprocessor config:

**/etc/snort/snort.conf**

---

```
preprocessor frag3_global: max_fragments 65536
```

```
preprocessor frag3_engine: policy windows detect_anomalies overlap_limit 10  
min_fragment_length 100 timeout 180
```

---

**preprocessor frag3\_global :** Configures global limits for IP fragmentation tracking to prevent resource exhaustion in Snort.

**max\_fragments 65536 :** Limits the maximum number of IP fragments Snort can track at one time across all network traffic to protect memory and performance.

**preprocessor frag3\_engine :** Enables and configures the IP fragment reassembly engine used for normalization and evasion detection.

**policy windows:** Reassembles IP fragments using Microsoft Windows TCP/IP stack behavior to match how the destination host processes packets.

**detect\_anomalies** : Detects abnormal or malformed IP fragmentation patterns such as overlaps or missing fragments.

**overlap\_limit 10**: Sets the maximum number of overlapping fragments allowed before triggering an alert to prevent evasion attempts.

**min\_fragment\_length 100** : Flags very small IP fragments (less than 100 bytes) which are commonly used in fragmentation-based evasion attacks.

**timeout 180** : Discards incomplete fragment sets if all fragments are not received within 180 seconds to prevent memory exhaustion.

These frag3 settings are default values provided by Snort and can be tuned by modifying parameters such as max\_fragments, overlap\_limit, min\_fragment\_length, and timeout based on the network infrastructure, traffic volume, and system performance requirements.

## **Stream5 preprocessor:**

Stream5 is Snort's transport-layer preprocessor that analyzes and normalizes transport-layer traffic before rule evaluation.

It performs TCP stream reassembly by joining split TCP packets using sequence numbers, so data sent across multiple packets is reconstructed as a single continuous stream. This prevents attackers from hiding malicious payloads by dividing them across several packets. Stream5 also reorders out-of-order TCP segments and handles retransmissions, ensuring the IDS sees the data exactly as the destination system would.

In addition, it detects overlapping TCP segments and abnormal TCP behavior such as data sent before a connection is established, which are commonly used to evade intrusion detection systems.

Stream5 tracks and detects half-open TCP connections, which occur when a connection handshake is initiated but never completed. In such cases, an attacker repeatedly initiates new TCP handshakes without completing the previous ones, leaving many connections in an incomplete (half-open) state. By maintaining TCP session state, Stream5 enables detection rules to identify this abnormal behavior.

Stream5 supports a wide range of TCP processing operations beyond those discussed here; additional TCP processing capabilities are documented in the official Stream5 README provided by Snort

In Snort's configuration, Stream5 provides extensive preprocessing capabilities for TCP, with limited UDP support through timeout-based flow tracking and ICMP stream tracking disabled by default due to its stateless nature.



## Stream5   preprocessor   Default   Snort   Preprocessor

### config:/etc/snort/snort.conf

```
preprocessor stream5_global: track_tcp yes, \
  track_udp yes, \
  track_icmp no, \
  max_tcp 262144, \
  max_udp 131072, \
  max_active_responses 2, \
  min_response_seconds 5
```

```
preprocessor stream5_tcp: log_asymmetric_traffic no, policy windows, \
  detect_anomalies, require_3whs 180, \
  overlap_limit 10, small_segments 3 bytes 150, timeout 180, \
  ports client 80 8080
preprocessor stream5_udp: timeout 180
```

**stream5\_global** : global flow-tracking behavior and resource limits for Stream5

**track\_tcp yes , track\_udp yes , track\_udp no** : Enables flow tracking for UDP TCP or ICMP traffic

**max\_tcp and max\_udp** : Maximum number of TCP and UDP sessions Stream5 can track at the same time

**max\_active\_responses 2, min\_response\_seconds 5** : Snort may send TCP RST packets to terminate suspicious connections, These two options limit how many TCP RST responses Snort can send and how often it can send them, in order to prevent response flooding

**stream5\_tcp** : Controls how TCP streams are tracked, reassembled, normalized, and checked for anomalies.

**log\_asymmetric\_traffic** : no Do not log traffic where only one side of the TCP conversation is seen

**require\_3whs 180** : Requires a valid 3-way handshake before treating traffic as established (waits up to 180 seconds).

**small\_segments 3 bytes 150** : Detects evasion using many tiny TCP segments

**ports client 80 8080** : Treats traffic to ports 80 and 8080 as **client-initiated** connections (In default snort connection lots of port no where mentioned)

**stream5\_udp** : Controls UDP flow

**timeout 180** : Expires UDP flows after 180 seconds of inactivity.

## http\_inspect preprocessor

HTTP Inspect is Snort's application-layer preprocessor that understands HTTP and normalizes it so attackers cannot hide payloads using encoding, compression, or malformed requests.

```
preprocessor http_inspect: global iis_unicode_map unicode.map 1252
compress_depth 65535 decompress_depth 65535 max_gzip_mem 104857600
preprocessor http_inspect_server: server default \
  http_methods { GET POST } \
  chunk_length 500000 \
  server_flow_depth 0 \
  client_flow_depth 0 \
  post_depth 65495 \
  oversize_dir_length 500 \
  max_header_length 750 \
  max_headers 100 \
  max_spaces 200 \
  small_chunk_length { 10 5 } \
  ports { 80 8080 } \
  non_rfc_char { 0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 } \
  enable_cookie \
  extended_response_inspection \
  inspect_gzip \
  normalize_utf \
  unlimited_decompress \
  normalize_javascript \
  apache_whitespace no \
  ascii no \
  bare_byte no \
  directory no \
  double_decode no \
  iis_backslash no \
  iis_delimiter no \
  iis_unicode no \
  multi_slash no \
  utf_8 no \
  u_encode yes \
  webroot no
```

**Http\_inspect: global**

Global HTTP decoding, normalization, and resource limits.

**iis\_unicode\_map unicode.map 1252**

Decodes IIS-style Unicode encoding.

**compress\_depth, decompress\_depth**

Controls inspection depth for compressed HTTP data.

**max\_gzip\_mem**

Limits memory usage during gzip decompression.

**http\_inspect\_server: server default**

Defines server-side HTTP inspection and normalization rules.

**http\_methods { ... }**

Specifies allowed HTTP methods.

**chunk\_length**

Limits HTTP chunk size to prevent evasion.

**server\_flow\_depth, client\_flow\_depth**

Controls how much HTTP request and response data is inspected.

**post\_depth**

Limits inspected HTTP POST body size.

**oversize\_dir\_length**

Detects abnormally long URL directory paths.

**max\_header\_length, max\_headers, max\_spaces :** Restrict HTTP header size, count, and whitespace.

**small\_chunk\_length { 10 5 }**

Detects excessive small chunked HTTP transfers.

**ports { 80 8080 443 }**

Defines ports on which HTTP traffic is inspected.

**non\_rfc\_char { ... }**

Flags non-RFC-compliant characters in HTTP traffic.

**enable\_cookie**

Enables HTTP cookie inspection.

**extended\_response\_inspection**

Enables deeper inspection of HTTP responses.

**inspect\_gzip**

Enables inspection of gzip-compressed HTTP content.

**normalize\_utf, normalize\_javascript, u\_encode yes**

Normalizes encoded HTTP data to prevent obfuscation.

**unlimited\_decompress**

Allows full decompression for complete inspection.

Server-specific normalization options

(apache, IIS, decoding controls)

Match server-specific HTTP parsing behavior to reduce false positives.

## What is gen-msg.map?

Snort preprocessors and decoders detect anomalies internally, but these detections are not displayed as alerts unless a corresponding rule wrapper is defined and loaded.

Such rule wrappers are written in .rules files using a special syntax that specifies the Generator ID (GID), Signature ID (SID), message, and classification.

When the internal GID–SID event matches a loaded rule wrapper, Snort displays the alert. If no rule wrapper is configured, the preprocessor continues to operate silently in the background while Snort proceeds with signature-based rule detection.

The gen-msg.map file is used by Snort to store Generator ID (GID) and Signature ID (SID) values along with their message descriptions. Each entry explains what a particular preprocessor or decoder event means. The GID and SID values that are used in preprocessor rule wrappers are taken from this file, so that internal Snort events can be shown as readable alert messages when a matching rule is written in a .rules file.

The gen-msg.map file is usually located in the same configuration directory as the snort.conf file (for example, /etc/snort/)

In this lab environment, default Snort rule files are used for preprocessor and decoder alerts instead of creating custom rules, as the goal is to understand how internal detections are mapped and displayed rather than to modify Snort's internal logic.

## Basic Components of a Snort Rule

- **Gid** – Identifies alerts generated internally by the Snort engine, primarily from preprocessors. GID is used to bind internal engine or preprocessor events to user-defined rules so that the event is raised as an alert. Specific GIDs and their corresponding alert IDs and messages can be referenced from the `gen-msg.map` file.
- **Action** – Defines what Snort should do when the rule is triggered. Common actions include alert, log, pass, drop, reject, etc.
- **Protocol** – Specifies the protocol the rule applies to, such as TCP, UDP, ICMP
- **Flags** – Used to match specific TCP flags in packets. This can help identify connection states.  
Examples: S (SYN), A (ACK), SA (SYN-ACK)
- **Source and Destination IP/Ports** – Indicates which IP addresses and ports the rule will inspect. Wildcards like `any` can be used.
- **content** – Searches for specific strings or payloads in packet data. Often used for detecting known attack patterns.  
Example: `content:"/etc/passwd";`
- **nocase** – Makes the content match case-insensitive. Useful when attackers try to bypass detection using casing variations.  
Example: `content:"select"; nocase;`
- **detection\_filter** – Controls how frequently a rule can trigger from a specific source. Helps in detecting brute-force or scanning behavior while reducing false positives.  
Example: `detection_filter:track by_src, count 5, seconds 60;`

- **sid** – The Snort rule ID. It uniquely identifies the rule and is required for every rule.

Example: sid:10000001;

- **rev** – The revision number of the rule. Used to keep track of rule updates.

Example: rev:1;

- **Classtype** – classtype is a predefined Snort classification used to label alerts by attack category and severity, helping prioritize incidents.

- **http\_uri** – Searches normalized HTTP request URIs, enabling detection of encoded or obfuscated URI-based attacks.

- **http\_client\_body** – Inspects normalized and decompressed HTTP request body content for malicious patterns.

- **http\_cookie** – Inspects normalized HTTP Cookie headers to detect cookie-based attacks, such as cookie injection, session fixation, session hijacking attempts, and manipulation of session identifiers.

For in-depth documentation, installation guides, and official rule writing tutorials, refer to the official Snort website: <https://docs.snort.org/>

## Overview of Some Rules I Created and used:

In this section, I have explained a selection of key Snort rules I wrote during this project. Rather than listing every rule line-by-line, I've highlighted a few that demonstrate different detection techniques used to monitor and identify specific attack behaviors. Each rule is shown with its purpose, logic, and configuration choices. This helps provide insight into how these rules were built not just for detection, but to minimize false positives while simulating real-world attack scenarios. All custom Snort rules written and used in this project to detect specific attacks are publicly available on my GitHub.

➤ **GitHub Repository:**

[https://github.com/prakashjs/project\\_with\\_snort\\_IDS](https://github.com/prakashjs/project_with_snort_IDS)



## Rule Definitions for Preprocessor-generated:

```
alert ( msg:"<Description of the preprocessor alert>";  
gid:<Generator_ID>;sid:<Signature_ID>;rev:1;metadata:<rule metadata>;  
classtype:<classification>;)
```

These rules are used to **surface and manage alerts generated internally by the Snort engine**, primarily from preprocessors. Unlike traditional rule-based detection, these alerts do not rely on packet content matching. Instead, they are triggered when a Snort preprocessor detects **protocol anomalies, decoding errors, or abnormal traffic behavior**.

The `gid` (Generator ID) identifies the internal Snort component that generated the alert, while the `sid` corresponds to a predefined event defined in the `gen-msg.map` file. By defining a GID-based rule, the internally generated event is promoted to a user-visible alert and can be classified, logged, or managed like a standard Snort rule.

## Rule Definitions for Port Scanning and DOS:

```
alert udp $EXTERNAL_NET any -> $HOME_NET ![53,5353] (detection_filter:track by_src, count 2, seconds 60; msg:"Potential UDP Port Scan Detected"; sid:10000001; rev:1;)

alert tcp $EXTERNAL_NET any -> $HOME_NET ![53,5353,80,443] (detection_filter:track by_src, count 2, seconds 60; msg:"Potential TCP Port Scan Detected"; sid:10000004; rev:1;)
```

The above rules were used to detect port scanning activity by monitoring for connection attempts to any TCP or UDP port other than TCP 80 (HTTP) and UDP ports 53, 5353 (DNS and mDNS). If more than 2 connection attempts occur from the same source IP within 60 seconds, an alert is triggered with the message *"Potential TCP/UDP Port Scan Detected."*

I chose not to use any flags in these rules. While it's common to use flags like S (SYN), A (ACK), or SA (SYN-ACK) and others to identify the state of TCP handshakes, I focused instead on simple connection attempt detection across disallowed ports.

The same rule structure was also used to detect Denial of Service (DoS) attacks, such as TCP flood and UDP flood. To achieve this, I changed the destination port to any, and set the detection threshold to count 1000, seconds 1. This allowed the rule to trigger an alert when a source sends 1000 or more packets within 1 second, which is typical flood behavior.

## Snort rule for Payload Based Attacks:

The following Snort rule format was used to detect various payload-based attacks, including **SQL Injection (SQLi)**, **Cross-Site Scripting (XSS)**, **Command Injection**, and **Path Traversal**:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (detection_filter: track by_src, count 3,
seconds 20;flow:to_server; msg:"<Potential attack detected>"; <http_specific_options>;
content:"<payload>";nocase;sid:<ID>;classtype:<Classtype>;rev:1;)
```

Each rule is designed to trigger an alert if three matching payloads are sent from the same source within 20 seconds, indicating potential automated or brute-force attack behavior.

The flow:to\_server; keyword ensures that only client-to-server traffic is inspected, which is where these types of injection attacks are typically delivered.

The content:"<payload>"; field is replaced with actual known malicious patterns to detect specific attack types.

The nocase; option is used to make the detection case-insensitive, improving effectiveness since attackers often obfuscate payloads using different character cases.

**HTTP-specific keywords (http\_uri, http\_client\_body, http\_cookie, etc.) –**

These options define **which part of the HTTP request is inspected**. By specifying an HTTP keyword, the rule is restricted to analyze only the relevant HTTP component.

This rule structure allows the detection of common web-based attack attempts by inspecting HTTP request bodies or parameters for signs of exploitation.

By keeping the rule generic and modifying only the msg and content fields, This approach made it easy to detect different types of attacks using the same basic rule format.

# Automation Tool Detection Using HTTP Headers

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (msg:"automation Scanner Detected";flow:to_server,established;content:"<Particular Header>"; http_header; nocase;classtype:attempted-recon;sid:10000126; rev:1;)
```

Many automated web scanners identify themselves through the HTTP User-Agent header when sending requests to a target web server. Security tools such as Nikto, Gobuster, FFUF, and SQLMap often include their tool name or version information within this header by default.

To detect such reconnaissance activity, I created Snort rules that inspect the HTTP User-Agent field using the `http_header` keyword. By matching known scanner identifiers within the header, Snort can generate alerts whenever these tools attempt to enumerate directories, discover vulnerabilities, or gather information about the target web application.

This detection method is effective against default scanner configurations and provides early visibility into reconnaissance activity. However, since attackers can modify or spoof the User-Agent header, this approach is best combined with behavior-based detection techniques such as monitoring abnormal request rates, directory enumeration patterns, and repeated access to sensitive resources.

## Snort rule for Malicious File Upload Detection:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 80 ( msg:"<Unauthorized File Type Upload Attempt>"; flow:to_server,established; content:"/DVWA/vulnerabilities/upload/"; http_uri; content:"<File_Extension>"; http_client_body; nocase; classtype:web-application-attack; sid:<SID>; rev:1;)
```

To identify potentially dangerous file upload attempts, I created a Snort rule that monitors HTTP requests sent to the DVWA File Upload page (/DVWA/vulnerabilities/upload/). The **rule inspects the uploaded file information contained within the HTTP request body and searches for file extensions** commonly associated with executable scripts or malicious content.

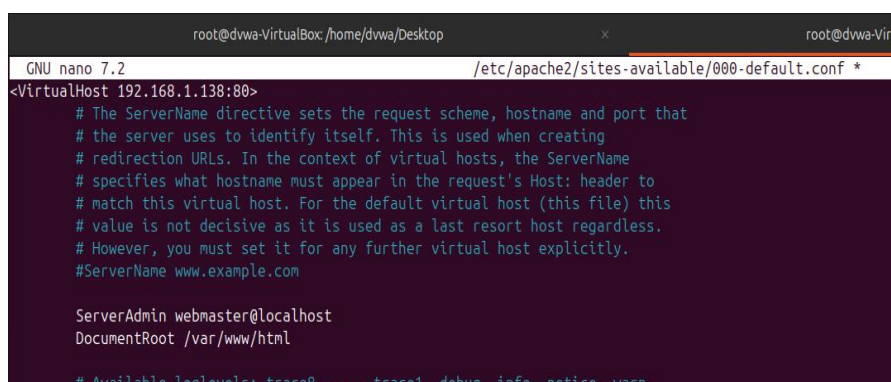
The detection logic focuses on identifying files such as .php, .jsp, .asp, .aspx, .exe, and other potentially dangerous formats that could be used to gain unauthorized access or execute commands on the web server. Since the DVWA upload functionality is intended to accept image files such as JPG, JPEG, and PNG, the presence of other file types may indicate an attempt to bypass upload restrictions or upload a web shell.

The rule uses **http\_uri** to ensure that only requests targeting the file upload page are inspected, while **http\_client\_body** is used to analyze the uploaded file information contained within the HTTP request body. When a suspicious file extension is detected, Snort generates an alert indicating a potential malicious file upload attempt.

# Apache Configuration Update

To ensure Apache listens only on the **web-facing interface** (higher metric), I updated the Apache configuration file located at `/etc/apache2/sites-available/000-default.conf` by changing the first line from

`<VirtualHost *:80>` to `<Web-Server-IP:80>` to ensure Apache listens only on the desired interface with the higher metric.



```
root@dwva-VirtualBox: /home/dwva/Desktop
GNU nano 7.2 /etc/apache2/sites-available/000-default.conf *
<VirtualHost 192.168.1.138:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
```

Additionally, the Apache port binding configuration located at `/etc/apache2/ports.conf` was modified by changing `Listen 80` to `Listen 192.168.1.141:80`.



```
root@dwva-VirtualBox: /home/dwva/Desktop
# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 192.168.1.141:80

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>
```

This ensures that Apache accepts HTTP connections only on the designated web-facing interface instead of listening on all available network interfaces.

## DVWA with Snort IDS Environment Setup:

- DVWA is deployed on an Ubuntu virtual machine.
- Web Server: Apache2 (apache2-bin version 2.4.58-1ubuntu8.5)
- Server IP & Port: 192.168.1.141 on port 80
- Snort is installed and configured on the same VM with custom rules.
- The virtual machine is configured with two network interfaces. The web server is bound to the interface with the higher routing metric, whereas the interface with the lower routing metric handles the operating system's primary network communication and default routing.
- Two separate instances of Snort were deployed, each using an independent configuration file. The first instance was dedicated to Snort's built-in preprocessors and default detection capabilities, while the second instance was configured to monitor traffic using custom rules developed for this project. This separation allowed independent analysis of preprocessor-generated alerts and user-defined rule alerts, simplifying testing, validation, and performance evaluation.

## Attack Simulation:

To simulate real-world attack scenarios, multiple offensive techniques were launched from a Kali Linux attacker machine against the target environment. These tests included :

- Malicious File Upload Attack
- Slowloris Denial-of-Service (DoS) Attack
- Fragmentation Evasion Attack
- Automated Scanning Tool Detection Using HTTP Headers (FFUF)
- TCP FIN Scan Detection
- Oversized HTTP Header Attack
- URL-Encoded Payload Attack

Attacker machine IPs : 192.168.1.147

## Objective:

The main objective is to trigger alerts in Snort when any of these attacks are attempted against DVWA.

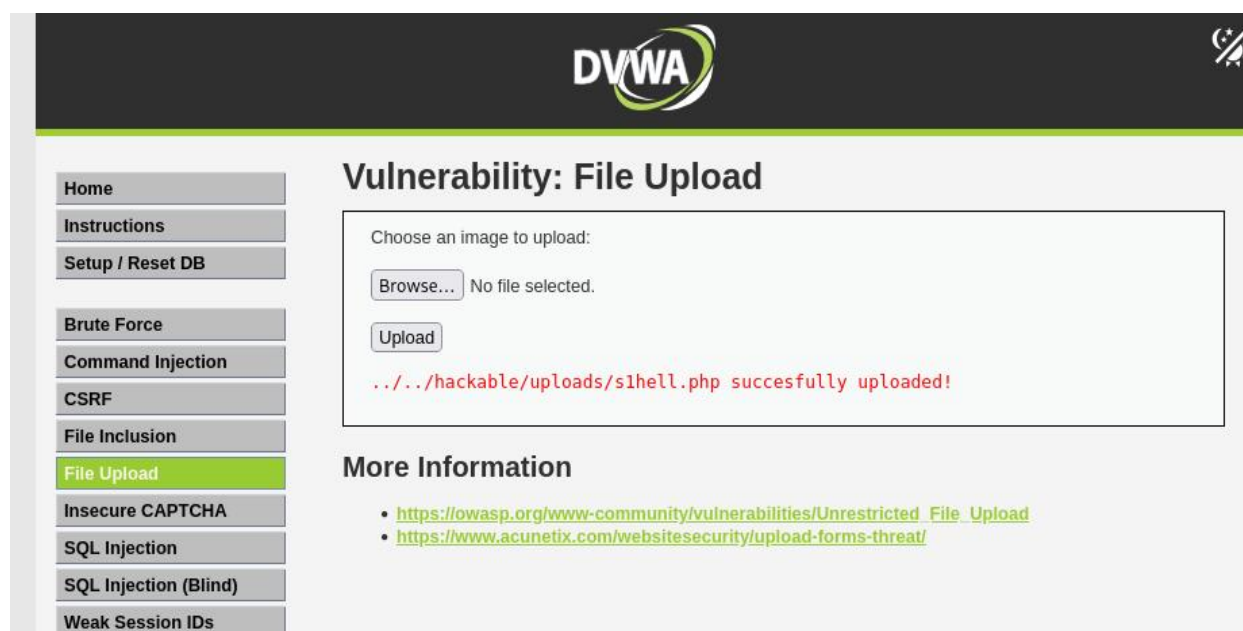
**Note :** Snort is capable of much more , this project focuses on writing custom rules to demonstrate detection capabilities for a selected set of common threats.



## File Upload Attack:

In this attack, a PHP file was uploaded to the web application to simulate a malicious file upload attempt. The purpose was to test the application's file validation mechanisms and evaluate Snort's ability to detect executable files being transferred through HTTP requests.

```
root@dvwa-VirtualBox: /etc/snort
root@dvwa-VirtualBox: /home/dvwa
root@dvwa-VirtualBox:/etc/snort# snort -c snort_custom.conf -A console -i enp0s8 -q
06/01-00:56:00.933960  [**] [1:10000132:1] Potential PHP File Upload [**] [Priority: 0] {TCP} 192.168.1.147:54466 -> 192.168.1.141:80
```



When the PHP file was uploaded, Snort generated an alert based on the custom detection rule configured for suspicious file extensions.

# Slowloris Attack Detection Using Snort Community

## Preprocessor Rules:

A Slowloris denial-of-service attack was simulated using the Slowloris tool against the DVWA web server. The attack established multiple TCP connections and periodically transmitted partial HTTP headers in an attempt to keep server connections open and exhaust available resources.

Snort was configured with community preprocessor rules and successfully detected the attack. During the attack, Snort generated the alert: **“Consecutive TCP small segments exceeding threshold”**.

```
root@dva-VirtualBox:/etc/snort# snort -c snort_preprocessor.conf -A console -i enp0s8 -q
06/01-01:09:14.155251 00000000 [129:12:1] Consecutive TCP small segments exceeding threshold [**] [Classification: Potentially Bad Traffic] [Priority: 2]
TCP 192.168.1.147:59278 -> 192.168.1.141:80
06/01-01:09:14.155253 00000000 [129:12:1] Consecutive TCP small segments exceeding threshold [**] [Classification: Potentially Bad Traffic] [Priority: 2]
TCP 192.168.1.147:59260 -> 192.168.1.141:80
06/01-01:09:14.155254 00000000 [129:12:1] Consecutive TCP small segments exceeding threshold [**] [Classification: Potentially Bad Traffic] [Priority: 2]
TCP 192.168.1.147:59274 -> 192.168.1.141:80
06/01-01:09:14.155254 00000000 [129:12:1] Consecutive TCP small segments exceeding threshold [**] [Classification: Potentially Bad Traffic] [Priority: 2]
TCP 192.168.1.147:59318 -> 192.168.1.141:80
06/01-01:09:14.155255 00000000 [129:12:1] Consecutive TCP small segments exceeding threshold [**] [Classification: Potentially Bad Traffic] [Priority: 2]
TCP 192.168.1.147:59316 -> 192.168.1.141:80
06/01-01:09:14.155255 00000000 [129:12:1] Consecutive TCP small segments exceeding threshold [**] [Classification: Potentially Bad Traffic] [Priority: 2]
TCP 192.168.1.147:59282 -> 192.168.1.141:80
06/01-01:09:14.155255 00000000 [129:12:1] Consecutive TCP small segments exceeding threshold [**] [Classification: Potentially Bad Traffic] [Priority: 2]
TCP 192.168.1.147:59250 -> 192.168.1.141:80
06/01-01:09:14.155256 00000000 [129:12:1] Consecutive TCP small segments exceeding threshold [**] [Classification: Potentially Bad Traffic] [Priority: 2]
TCP 192.168.1.147:59284 -> 192.168.1.141:80
06/01-01:09:14.249883 00000000 [129:12:1] Consecutive TCP small segments exceeding threshold [**] [Classification: Potentially Bad Traffic] [Priority: 2]
TCP 192.168.1.147:59300 -> 192.168.1.141:80
06/01-01:09:14.275500 00000000 [129:12:1] Consecutive TCP small segments exceeding threshold [**] [Classification: Potentially Bad Traffic] [Priority: 2]
TCP 192.168.1.147:59294 -> 192.168.1.141:80
```

```
(root@pirate)-[/var/www/html]
# slowloris -p 80 -s 10 192.168.1.141
[01-06-2026 01:08:36] Attacking 192.168.1.141 with 10 sockets.
[01-06-2026 01:08:36] Creating sockets...
[01-06-2026 01:08:40] Sending keep-alive headers...
[01-06-2026 01:08:40] Socket count: 0
[01-06-2026 01:08:40] Creating 10 new sockets...
[01-06-2026 01:08:59] Sending keep-alive headers...
[01-06-2026 01:08:59] Socket count: 0
[01-06-2026 01:08:59] Creating 10 new sockets...
[01-06-2026 01:09:14] Sending keep-alive headers...
[01-06-2026 01:09:14] Socket count: 10
```

This alert was produced by the Stream5 preprocessor, which monitors TCP session behavior and identifies suspicious patterns such as excessive numbers of small TCP segments commonly associated with Slowloris-style attacks.

The successful detection demonstrates the effectiveness of Stream5 in identifying TCP-based denial-of-service techniques and session-layer evasion behaviors that may not be detected by traditional signature-based rules alone.

## Fragmentation Evasion Detection Using Frag3:

To evaluate Snort's IP fragmentation detection capabilities, a fragmented scan was performed using Nmap with a small MTU value (`nmap --mtu 8`). The generated traffic produced multiple tiny IP fragments intended to evade packet inspection by splitting packets into unusually small fragments.

The Frag3 preprocessor was configured with the following parameters:

**preprocessor frag3\_engine: policy windows detect\_anomalies overlap\_limit 10 min\_fragment\_length 100 timeout 180**

The `min_fragment_length` option was set to **100 bytes**, meaning that any fragment smaller than 100 bytes would be treated as anomalous and inspected for potential evasion attempts. During testing, the attack generated fragments approximately **8 bytes** in size, which fell well below the configured threshold.

```
# nmap -mtu 8 -p 21,22,8080,123,161,110 192.168.1.141
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-01 03:47 +0530
Nmap scan report for 192.168.1.141
Host is up (0.065s latency).

PORT      STATE SERVICE
21/tcp    closed ftp
22/tcp    closed ssh
110/tcp   closed pop3
123/tcp   closed ntp
161/tcp   closed snmp
8080/tcp  closed http-proxy
```

```
root@dvwa-VirtualBox:/etc/snort# snort -c snort_preprocessor.conf -A console -i enp0s8 -q
06/01-01:27:05.970340 192.168.1.141 192.168.1.141 [**] [123:13:1] (spp_frag3) Tiny fragment [**] [Classification: Attempted Denial of Service] [Priority: 2] (TC
06/01-01:27:05.970340 192.168.1.141 192.168.1.141 [**] [123:13:1] (spp_frag3) Tiny fragment [**] [Classification: Attempted Denial of Service] [Priority: 2] (TC
06/01-01:27:05.970341 192.168.1.141 192.168.1.141 [**] [123:13:1] (spp_frag3) Tiny fragment [**] [Classification: Attempted Denial of Service] [Priority: 2] (TC
06/01-01:27:05.970341 192.168.1.141 192.168.1.141 [**] [123:13:1] (spp_frag3) Tiny fragment [**] [Classification: Attempted Denial of Service] [Priority: 2] (TC
```

As a result, Snort's Frag3 preprocessor successfully detected the activity and generated the alert: **(spp\_frag3) Tiny fragment**

This alert indicates that the observed fragments violated the configured minimum fragment length threshold. Frag3 analyzed the fragmented traffic and identified the packets as suspicious, demonstrating its ability to detect fragmentation-based evasion techniques that attackers may use to bypass traditional packet inspection mechanisms.

## Automated Scanning Tool Detection Using HTTP Headers:

To evaluate the custom rule's ability to detect automated web enumeration activity, a directory discovery scan was performed using FFUF against the DVWA web application. FFUF is a web fuzzing tool that generates numerous HTTP requests to identify hidden files, directories, APIs, and other web resources using a supplied wordlist.

```
(root@pirate)~[/home/dillinger]
# ffuf -u http://192.168.1.141/DVWA/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200

v2.1.0-dev

-----
:: Method      : GET
:: URL         : http://192.168.1.141/DVWA/FUZZ
:: Wordlist     : FUZZ: /usr/share/wordlists/dirb/common.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200
-----

.git/HEAD      [Status: 200, Size: 23, Words: 2, Lines: 2, Duration: 31ms]
favicon.ico   [Status: 200, Size: 1406, Words: 5, Lines: 2, Duration: 13ms]
php.ini       [Status: 200, Size: 154, Words: 19, Lines: 6, Duration: 7ms]
```

```
Jun 3 05:43
root@dvwa-VirtualBox: /etc/snort

root@dvwa-VirtualBox: /etc/snort
root@dvwa-VirtualBox: /etc/snort/rules

0 -> 192.168.1.141:80
6/03-05:42:15.590460 192.168.1.141:80 [**] [1:10000126:2] FFUF Scanner Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {
0 -> 192.168.1.141:80
6/03-05:42:15.600099 192.168.1.141:80 [**] [1:10000126:2] FFUF Scanner Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {
2 -> 192.168.1.141:80
6/03-05:42:15.600014 192.168.1.141:80 [**] [1:10000126:2] FFUF Scanner Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {
2 -> 192.168.1.141:80
6/03-05:42:15.603258 192.168.1.141:80 [**] [1:10000126:2] FFUF Scanner Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {
0 -> 192.168.1.141:80
6/03-05:42:15.600014 192.168.1.141:80 [**] [1:10000126:2] FFUF Scanner Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {
6 -> 192.168.1.141:80
6/03-05:42:15.608738 192.168.1.141:80 [**] [1:10000126:2] FFUF Scanner Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {
4 -> 192.168.1.141:80
6/03-05:42:15.617615 192.168.1.141:80 [**] [1:10000126:2] FFUF Scanner Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {
4 -> 192.168.1.141:80
6/03-05:42:15.608829 192.168.1.141:80 [**] [1:10000126:2] FFUF Scanner Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {
2 -> 192.168.1.141:80
6/03-05:42:15.608736 192.168.1.141:80 [**] [1:10000126:2] FFUF Scanner Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {
6 -> 192.168.1.141:80
6/03-05:42:15.619480 192.168.1.141:80 [**] [1:10000126:2] FFUF Scanner Detected [**] [Classification: Attempted Information Leak] [Priority: 2] {
```

Since the custom rule was configured to use the "FUZZ" keyword within HTTP headers as a detection signature, any request containing this pattern would trigger an alert. During the FFUF scan, the tool generated HTTP requests containing the specified keyword, allowing Snort to successfully match the signature and identify the activity as automated web enumeration.

This demonstrates how custom signatures can be used to detect reconnaissance tools based on characteristic request patterns present in network traffic.



## TCP FIN Scan Detection Using Community Preprocessor Rules

To evaluate Snort's ability to detect transport-layer reconnaissance activities, a TCP FIN scan was performed against the target system using Nmap. Unlike a traditional TCP connection scan, a FIN scan sends packets with only the FIN flag set and does not complete the standard TCP three-way handshake.

```

# nmap -p 8080,22,21,123 -sF 192.168.1.141
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-04 09:21 +0530
Nmap scan report for 192.168.1.141
Host is up (0.057s latency).

PORT      STATE SERVICE
21/tcp    closed ftp
22/tcp    closed ssh
123/tcp   closed ntp
8080/tcp  closed http-proxy

```

```

root@dvw-VirtualBox:/etc/snort# snort -c snort_preprocessor.conf -A console -i enp0s8 -q
06/04-09:21:14.918964  [**] [129:20:1] TCP session without 3-way handshake [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.1
147:39894 -> 192.168.1.141:22
06/04-09:21:14.919106  [**] [129:20:1] TCP session without 3-way handshake [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.1
147:39894 -> 192.168.1.141:21
06/04-09:21:14.920375  [**] [129:20:1] TCP session without 3-way handshake [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.1
147:39894 -> 192.168.1.141:8080
06/04-09:21:14.921455  [**] [129:20:1] TCP session without 3-way handshake [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.1
147:39894 -> 192.168.1.141:123
06/04-09:21:22.283237  [**] [129:20:1] TCP session without 3-way handshake [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.1
147:45037 -> 192.168.1.141:21
06/04-09:21:22.283240  [**] [129:20:1] TCP session without 3-way handshake [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.1
147:45037 -> 192.168.1.141:8080
06/04-09:21:22.283621  [**] [129:20:1] TCP session without 3-way handshake [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.1
147:45037 -> 192.168.1.141:22
06/04-09:21:22.284194  [**] [129:20:1] TCP session without 3-way handshake [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.1
147:45037 -> 192.168.1.141:123

```

The purpose of this scan is to identify accessible services and gather information about the target host while generating less conventional network traffic. During the attack, FIN packets were transmitted to multiple destination ports on the target system in a short period of time.

Snort, using the community preprocessor rule set, successfully detected the scanning activity and generated alerts indicating suspicious port probing behavior. The detection was based on observing abnormal TCP traffic patterns across multiple destination ports, which are characteristic of reconnaissance and enumeration activities.

The generated alerts demonstrated Snort's ability to identify transport-layer scanning techniques before exploitation attempts occur. This experiment highlights the effectiveness of preprocessor-based detection mechanisms in recognizing network reconnaissance activity that may be used to map available services and identify potential attack vectors.

## Oversized HTTP Header Detection Using HTTP Inspect

To evaluate HTTP protocol anomaly detection capabilities, an oversized HTTP header attack was performed against the target web application. The attack involved sending an HTTP request containing a custom header whose length exceeded the maximum header length configured in the HTTP Inspect preprocessor.

```
(root@pirate)-[/home/dillinger]
# curl -H "X-TEST: $(python3 -c 'print("A"*1000)')' http://192.168.1.141/DVWA/index.php
(root@pirate)-[/home/dillinger]
#
```

```
root@dvwa-VirtualBox:/etc/snort# snort -c snort_preprocessor.conf -A console -i enp0s8 -q
06/05-06:58:24.832619  [**] [119:19:1] (http_inspect) LONG HEADER [**] [Classification: Potentially Bad Traffic] [Priority: 2]
-> 192.168.1.141:80
```

A request was generated with a header length greater than 750 bytes, violating the configured protocol limits. During inspection, HTTP Inspect analyzed the incoming request and identified the oversized header as anomalous HTTP traffic.

This test demonstrates the ability of HTTP Inspect to detect malformed or suspicious HTTP requests that exceed normal protocol limits. Such requests may be used by attackers to perform buffer overflow attempts, denial-of-service attacks, or HTTP protocol evasion techniques.





## Conclusion

This project demonstrated the implementation of Snort IDS to monitor and detect various attacks against the Damn Vulnerable Web Application (DVWA) in a controlled laboratory environment. Through the use of both Snort's built-in preprocessors and custom-written detection rules, it was possible to identify a range of malicious activities including port scanning, SQL injection, cross-site scripting (XSS), command injection, malicious file uploads, denial-of-service attacks, and reconnaissance attempts.

The project also highlighted the importance of traffic normalization through preprocessors such as Frag3, Stream5, and HTTP Inspect, which improve detection accuracy by exposing obfuscated or fragmented attacks. Overall, this work provided practical experience in intrusion detection, rule development, network traffic analysis, and security monitoring, while demonstrating how signature-based and anomaly-based detection techniques can be combined to enhance the security of web applications.

Future enhancements may include deploying Snort in IPS mode, integrating centralized logging and SIEM solutions, migrating to Snort 3 for improved performance and detection capabilities, and extending security monitoring to modern attack techniques.